

CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring



10.4 Connectivity

1. Paths

A *path of length n* in a **simple graph** is a sequence of vertices $v_0, v_1, \dots, v_n$ such that $\{v_0, v_1\}, \{v_1, v_2\}, \dots, \{v_{n-1}, v_n\}$ are n edges in the graph.

The path is a *circuit* if it begins and ends at the same vertex (length greater than 0).

A path is *simple* if it does not contain the same edge more than once.



Note:

1. There is nothing to prevent traversing an edge back and forth to produce arbitrarily long paths. This is usually not interesting which is why we define a simple path.
2. The notation of a path: vertex sequence
3. A path of length zero consists of a single vertex.

**Path in directed graph**

A **path of length n** in a **directed graph** is a sequence of vertices $v_0, v_1, \dots, v_n$ such that $(v_0, v_1), (v_1, v_2), \dots, (v_{n-1}, v_n)$ are n directed edges in the graph.

Circuit or cycle : the path begins and ends with the same vertex.

Simple path: the path does not contain the same edge more than once.



Paths represent useful information in many graph models.**[[Example 1]] Path in Acquaintanceship Graphs**

In an acquaintanceship graph there is a path between two people if there is a chain of people linking these people, where two people adjacent in the chain know one other.

Many social scientists have conjectured that almost every pair of people in the world are linked by a small chain of people, perhaps containing just five or fewer people.

The play *Six Degrees of Separation* by John Guare is based on this notion.

**2. Counting paths between vertices**

The number of paths between two vertices in a graph can be determined using its adjacency matrix.



【 Theorem 2 】 The *number of different paths of length r* from v_i to v_j is equal to the (i, j) th entry of A^r , where A is the adjacency matrix representing the graph consisting of vertices $v_1, v_2, \dots, v_n$.

Note: This is the standard power of A , not the Boolean product.

Proof:

Let $A = [a_{ij}]_{n \times n}$ (3)

(1) A

$$a_{ij} = 1$$

$$a_{ij} = 0$$

$$A^r = (c_{ij})_{n \times n}$$

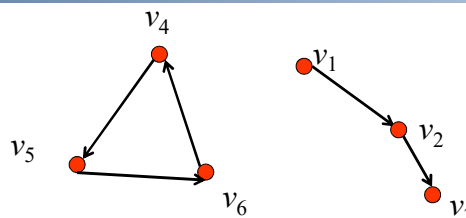
$$A^{r+1} = A^r \cdot A = (d_{ij})_{n \times n}$$

$$d_{ij} = c_{i1}a_{1j} + c_{i2}a_{2j} + \dots + c_{in}a_{nj} = \sum_{k=1}^n c_{ik}a_{kj}$$

from v_i to v_j



【 Example 2 】



(1) How many paths of length 2 are there from v_5 to v_4 ?

a_{54} in A^2 : 1

(2) The number of paths not exceeding 6 are there from v_4 to v_5 ?

a_{45} in $A + A^2 + A^3 + A^4 + A^5 + A^6$: 2

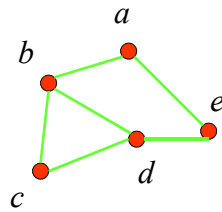
(3) The number of circuits starting at vertex v_5 whose length is not exceeding 6?



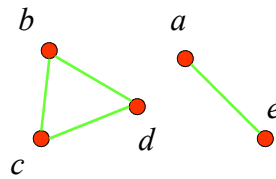
3. Connectedness in undirected graphs

An undirected graph is called *connected* if there is a path between **every pair of distinct vertices** of the graph.

[[Example 3]] Are the following graphs connected?



Yes.



No.



【 Theorem 1 】 There is a simple path between every pair of distinct vertices of a connected undirected graph.

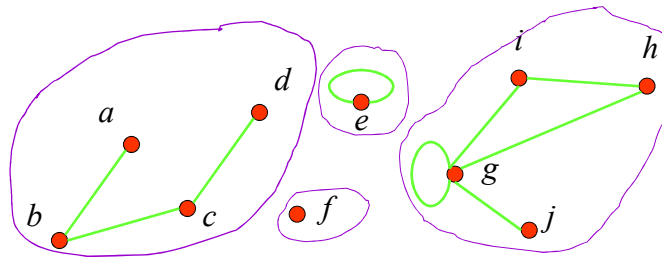
Proof:

Because the graph is connected there is a path between u and v . Throw out all redundant circuits to make the path simple.



The maximally connected subgraphs of G are called the *connected components* or just the *components*.

For example,

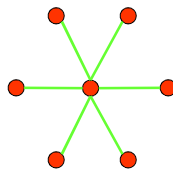


A vertex is a *cut vertex* (or *articulation point*), if removing it and all edges incident with it results in more connected components than in the original graph.

Similarly if removal of an edge creates more components the edge is called a *cut edge* or *bridge*.

For example,

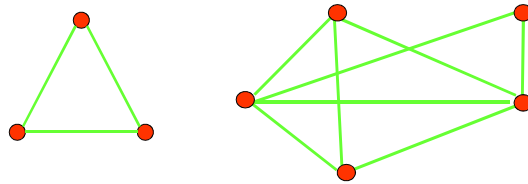
(1)



In the star network the center vertex is a cut vertex.
All edges are cut edges.



(2)



There are no cut edges or vertices in the graph G above.
Removal of any vertex or edge does not create additional components.



4. Connectedness in directed graphs

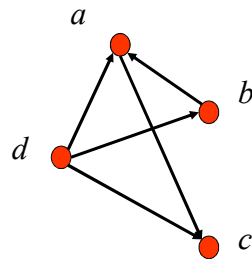
A directed graph is *strongly connected* if there is a path from a to b and from b to a for all vertices a and b in the graph.

The graph is *weakly connected* if the underlying undirected graph is connected.

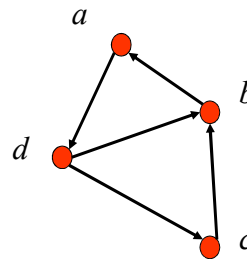
Note: By the definition, any strongly connected directed graph is also weakly connected.



For example,



Weakly connected

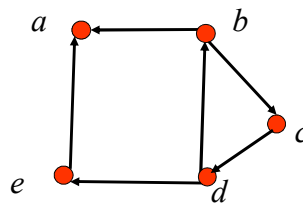


Strongly connected



For directed graph, the maximal strongly connected subgraphs are called the *strongly connected components* or just the *strong components*.

For example,



Problems:

1. How to determine whether a given directed graph is strongly connected or weakly connected ?
2. How to find the strongly connected components in a directed graph ?

Kosararu

Tarjan



5. Paths and Isomorphism

Idea:

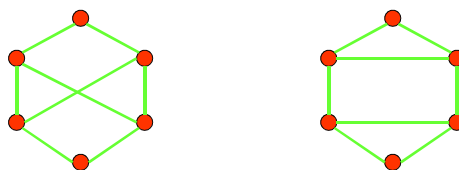
(1) Some other invariants

- The number and size of connected components
- Path
 - ✓ Two graphs are isomorphic only if they have simple circuits of the same length.
 - ✓ Two graphs are isomorphic only if they contain paths that go through vertices so that the corresponding vertices in the two graphs have the same degree.

(2) We can also use paths to find mapping that are potential isomorphism.



[[Example 4]] Are these two graphs isomorphic?



Solution:

No.

Because the right graph contains circuits of length 3, while the left graph does not.



Homework:

Sec. 10.4 27(e), 28, 29, 62



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

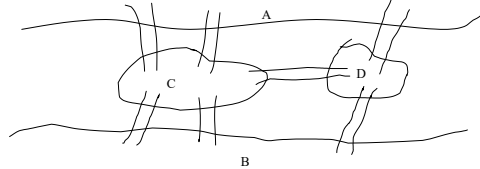
10.7 Planar Graphs

10.8 Graph Coloring



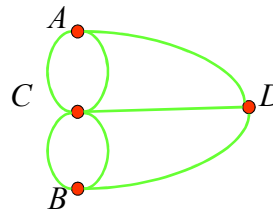
9.5.1 Euler Paths

1. Königsberg Seven Bridge Problem



Problem: Is it possible to start at some location in the town, travel across all the bridges without crossing any bridge twice, and return the starting point?

The question becomes: Is there a simple circuit in this multigraph that contains every edge?



Terminologies:

■ *Euler Path*

An Euler path is a **simple path** containing every edge of G .

■ *Euler Circuit*

An Euler circuit is a **simple circuit** containing every edge of G .

■ *Euler Graph*

A graph contains an Euler circuit.



2. Necessary and sufficient conditions for Euler circuit and paths

【 Theorem 1 】 A connected multigraph has an Euler circuit if and only if each of its vertices has even degree.

Proof:

(1) Necessary condition

G has an Euler circuit $\Rightarrow$ Every vertex in V has even degree

Consider the Euler circuit.

- ◆ the vertex a which the Euler circuit begins with
- ◆ intermeidate vertices



(2) sufficient condition

We will **form a simple circuit** that begins at an arbitrary vertex a of G .

- Build a simple circuit $x_0=a, x_1, x_2, \dots, x_n=a$.
- An Euler circuit has been constructed if all the edges have been used. otherwise,
- Consider the **subgraph H** from G by deleting the edges already uses and vertices that are not incident with any remaining edges.

Let w be a vertex which is the common vertex of the circuit and H .

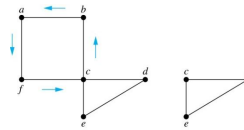
Beginning at w , construct a simple path in H .



Sufficient Conditions for Euler Circuits and Paths

Suppose that G is a connected multigraph with ≥ 2 vertices, all of even degree. Let $x_0 = a$ be a vertex of even degree. Choose an edge $\{x_0, x_1\}$ incident with a and proceed to build a simple path $\{x_0, x_1\}, \{x_1, x_2\}, \dots, \{x_{n-1}, x_n\}$ by adding edges one by one until another edge can not be added.

We illustrate this idea in the graph G here. We begin at a and choose the edges $\{a, f\}, \{f, c\}, \{c, b\}$, and $\{b, a\}$ in succession.

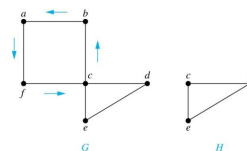


- The path begins at a with an edge of the form $\{a, x\}$; we show that it must terminate at a with an edge of the form $\{y, a\}$. Since each vertex has an even degree, there must be an even number of edges incident with this vertex. Hence, every time we enter a vertex other than a , we can leave it. Therefore, the path can only end at a .
- If all of the edges have been used, an Euler circuit has been constructed. Otherwise, consider the subgraph H obtained from G by deleting the edges already used.

➤ In the example H consists of the vertices c, d, e .



Sufficient Conditions for Euler Circuits and Paths (continued)



- Because G is connected, H must have at least one vertex in common with the circuit that has been deleted.

➤ In the example, the vertex is c .

- Every vertex in H must have even degree because all the vertices in G have even degree and for each vertex, pairs of edges incident with this vertex have been deleted. Beginning with the shared vertex construct a path ending in the same vertex (as was done before). Then splice this new circuit into the original circuit.

➤ In the example, we end up with the circuit a, f, c, d, e, c, b, a .

- Continue this process until all edges have been used. This produces an Euler circuit. Since every edge is included and no edge is included more than once.
- Similar reasoning can be used to show that a graph with exactly two vertices of odd degree must have an Euler path connecting these two vertices of odd degree.



Algorithm for Constructing an Euler Circuits

In our proof we developed this algorithms for constructing a Euler circuit in a graph with no vertices of odd degree.

```

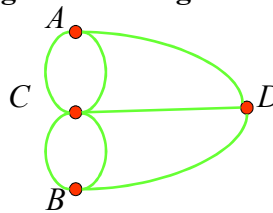
procedure Euler(G: connected multigraph with all vertices of even degree)
  circuit := a circuit in G beginning at an arbitrarily chosen vertex
  with edges successively added to form a path that returns to this
  vertex.
  H := G with the edges of this circuit removed
  while H has edges
    subcircuit := a circuit in H beginning at a vertex in H that also is
    an endpoint of an edge in circuit.
    H := H with edges of subcircuit and all isolated vertices
    removed
    circuit := circuit with subcircuit inserted at the
    appropriate vertex.
  return circuit{circuit is an Euler circuit}
  
```



10.5.1 Euler Paths

【 Theorem 2 】 A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree.

【 Example 1 】 Königsberg Seven Bridge Problem

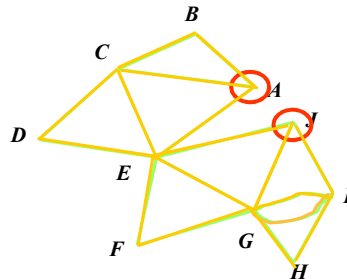


Solution:

1. The graph has four vertices of odd degree. Therefore, it does not have an Euler circuit.
2. It does not have an Euler path either.



[[Example 2]] Determine whether the following graph has an Euler path. Construct such a path if it exists.



The Euler path:

$A, C, E, F, G, I, J, E, A, B,$
 C, D, E, G, H, I, G, J

Solution:

The graph has 2 vertices of odd degree, and all of other vertices have even degree. Therefore, this graph has an Euler path.



3. Euler circuit and paths in directed graphs

A directed multigraph having no isolated vertices has an Euler circuit if and only if

- the graph is weakly connected
- the in-degree and out-degree of each vertex are equal.

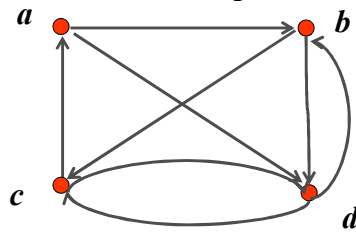
A directed multigraph having no isolated vertices has an Euler path but not an Euler circuit if and only if

- the graph is weakly connected
- the in-degree and out-degree of each vertex are equal

for all but two vertices, one that has in-degree 1 larger than its out-degree and the other that has out-degree 1 larger than its in-degree.



[[Example 3]] Determine whether the directed graph has an Euler path. Construct an Euler path if it exists.



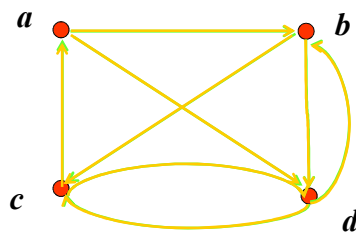
Solution:

	$\deg^-(v)$	$\deg^+(v)$
<i>a</i>	1	2
<i>b</i>	2	2
<i>c</i>	2	2
<i>d</i>	3	2

Hence, the directed graph has an Euler path.



[[Example 3]] Determine whether the directed graph has an Euler circuit. Construct an Euler circuit if it exists.



Solution:

	$\deg^-(v)$	$\deg^+(v)$
<i>a</i>	1	2
<i>b</i>	2	2
<i>c</i>	2	2
<i>d</i>	3	2

Hence, the directed graph has an Euler path.



4. Applications

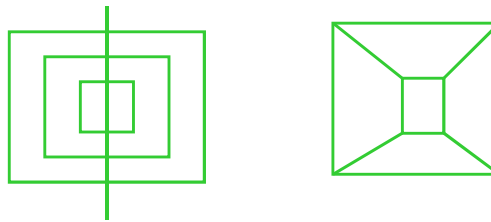
Euler path or circuit can be used to solve many practical problems.

1) A type of puzzle

Draw a picture in a continuous motion without lifting a pencil so that no part of the picture is retraced.

The equivalent problem: Whether the graph exist an Euler path or circuit.

For example,



4. Applications

Euler path or circuit can be used to solve many practical problems.

2) The Chinese postman problem

- The problem is named in honor of Guan Meigu (管梅谷), who posed it in 1962.

3) The other area, such as networking, molecular biology etc.



10.5.2 Hamilton paths and circuit

Hamilton's puzzle



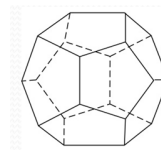
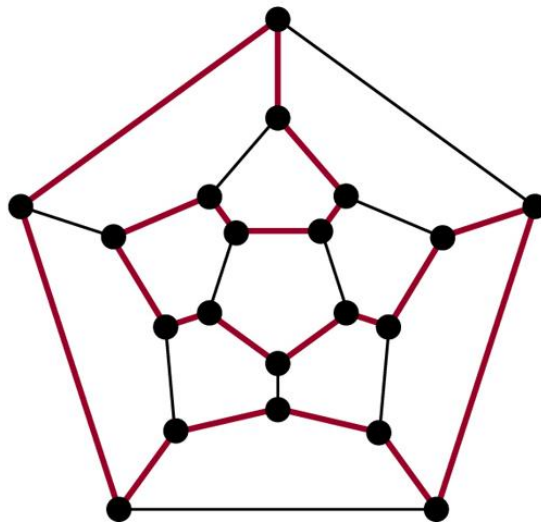
The object of the puzzle was to start at a city and travel along the edges of the dodecahedron, visiting each of the other 19 cities exactly once, and end back at the first city.

The equivalent question: Is there a circuit in the graph shown in b that passes through each vertex exactly once?

35



© The McGraw-Hill Companies, Inc. all rights reserved.



36



A *Hamilton path* in a graph G is a path which visits every vertex in G exactly once.

A *Hamilton circuit* (or *Hamilton cycle*) is a cycle which visits every vertex exactly once, *except for the first vertex*, which is also visited at the end of the cycle.

If a connected graph G has a Hamilton circuit, then G is called a *Hamilton graph*.

Question:

- 1) H path is a simple path?



1. The sufficient condition for the existence of Hamilton path and Hamilton circuit

【 Theorem 3 】 DIRAC'THEOREM

If G is a simple graph with n vertices with $n \geq 3$ such that the degree of every vertex in G is at least $n/2$, then G has a Hamilton circuit.

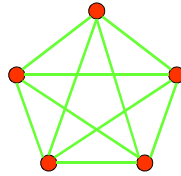
【 Theorem 4 】 ORE'THEOREM

If G is a simple graph with n vertices with $n \geq 3$ such that $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices u and v in G , then G has a Hamilton circuit.



[[Example 4]] Show that K_n has a Hamilton circuit whenever $n \geq 3$?

For example,



Proof:

We can form a Hamilton circuit in K_n beginning at any vertex. Such a circuit can be built by visiting vertices in any order we choose, as long as the path begins and ends at the same vertex and visits each other vertex exactly once. This is possible since there are edges in K_n between any two vertices.



2. The necessary condition for Hamilton path and Hamilton circuit

For undirected graph:

The necessary condition for the existence of Hamilton path:

- G is connected;
- There are at most two vertices which degree are less than 2.



2. The necessary condition for Hamilton path and Hamilton circuit

The necessary condition for the existence of Hamilton circuit:

- The degree of each vertex is larger than 1.

Some properties:

- If a vertex in the graph has degree two, then both edges that are incident with this vertex must be part of any Hamilton circuit.
- When a Hamilton circuit is being constructed and this circuit has passed through a vertex, then all remaining edges incident with this vertex, other than the two used in the circuit, can be removed from consideration.

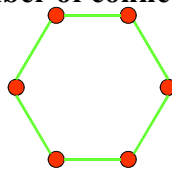


Another important necessary condition

- G is a Hamilton graph, for any nonempty subset S of set V , the number of connected components in $G-S \leq |S|$.

Note:

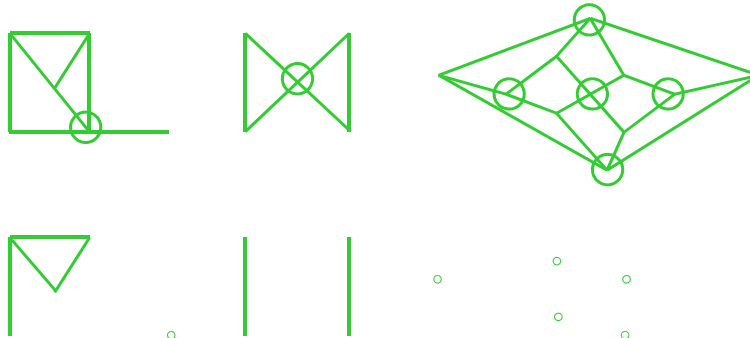
- (1) $G-S$ is a subgraph of G
- (2) Suppose that C is a H circuit of G . For any nonempty subset S of set V , the number of connected components in $C-S \leq |S|$.



- (3) the number of connected components in $G-S \leq$ the number of connected components in $C-S$



[[Example 5]] Does the following graphs have a Hamilton circuit?



3. Applications

Hamilton path or circuit can be used to solve many practical problems also.

For example,

- 1) Find a path or circuit that visits each road intersection in a city, or each node in a communication network exactly once.
- 2) The famous **traveling salesman problem (TSP)**
- 3)



【Example 6】 There are seven people denoted by A, B, C, D, E, F, G. Suppose that the following facts are known.

A--English (A can speak English.)

B--English, Chinese

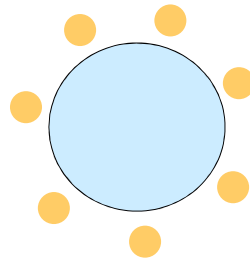
C--English, Italian, Russian

D--Japanese, Chinese

E--German, Italia

F--French, Japanese, Russian

G--French, German



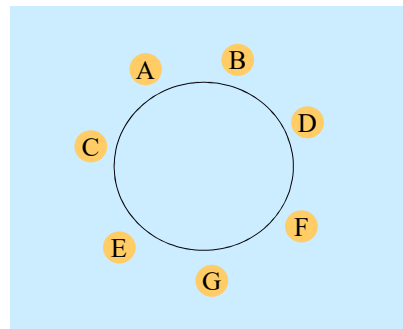
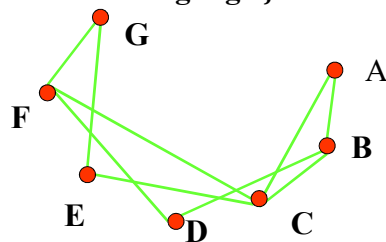
How to arrange seat for the round desk such that the seven people can talk each other?



Solution:

(1) Construct graph

$V = \{A, B, C, D, E, F, G\}$, $E = \{(u, v) | u, v \text{ can speak at least one common language.}\}$



(2) If there is a H circuit, then we can arrange seat for the round desk such that the seven people can talk each other.

H circuit: A,B,D,F,G,E,C,A



Homework:

Sec. 10.5 4, 6, 31, 34, 38, 41



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring

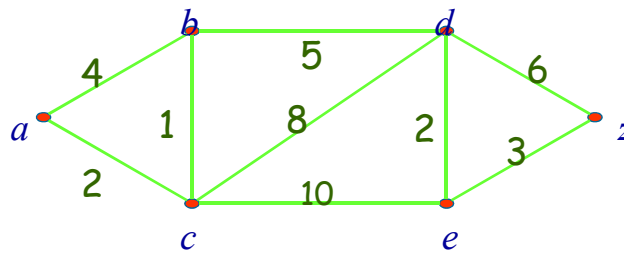


1. Introduction

■ *Weighted graph* $G = (V, E, W)$

We can assign weights to the edges of graphs.

For example,



1. Introduction

■ *Weighted graph* $G = (V, E, W)$

We can assign weights to the edges of graphs.

■ *The length of a path in a weighted graph*

-- the sum of the weights of the edges of this path.

■ *Shortest Path Problems*

$G = (V, E, W)$ is a weighted graph, where $w(x, y)$ is the weight of edge (x, y)
 (if $(x, y) \notin E$, $w(x, y) = \infty$). $a, z \in V$, find the shortest path between a and z .



2. A shortest path algorithm

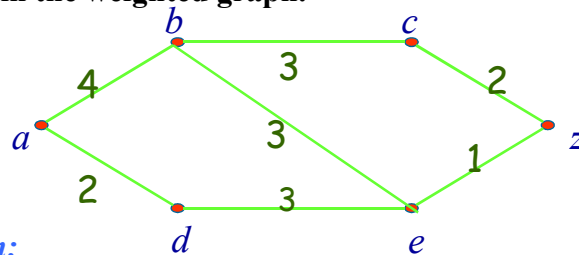
There are several different algorithms that find the shortest path between two vertices in a weighted graph.

Dijkstra's Algorithm (undirected graph with positive weights)

- ❑ An algorithm discovered by the Dutch mathematician E. Dijkstra in 1959.
- ❑ An iterative procedure.
- ❑ Proceed by finding the length of the shortest path from a to a first vertex, the length of the shortest path from a to a second vertex, and so on, until the length of the shortest path from a to z is found.



[[Example 1]] What is the length of the shortest path between a and z in the weighted graph.



Solution:

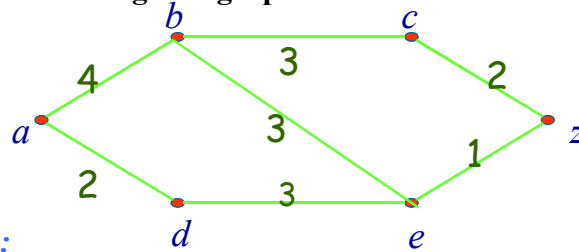
We will solve this problem by finding the length of a shortest path from a to successive vertices, until z is reached.

1) The first closest vertex: d

The only paths starting at a that contain no vertex other than a are a, b and a, d .



[[Example 1]] What is the length of the shortest path between a and z in the weighted graph.



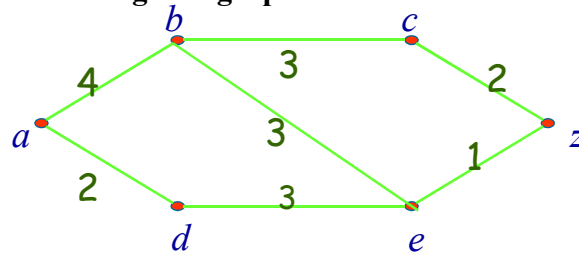
Solution:

- 1) The first closest vertex: d
- 2) The second closest vertex: b

Looking at all paths that go through only a and d .



[[Example 1]] What is the length of the shortest path between a and z in the weighted graph.



Solution:

- 1) The first closest vertex: d
- 2) The second closest vertex: b
- 3) The third closest vertex: e

Examine only paths go through only a , d and b .

- 4) The forth closest vertex: z



The details of Dijkstra's algorithm

Dijkstra's algorithm proceeds by forming a distinguished set of vertices iteratively. Let S_k denote this set of vertices after k iterations of **labeling procedure**.

Step 1 Label a with 0 and other with ∞ , i.e. $L_0(a)=0$, and $L_0(v)=\infty$ and $S_0=\emptyset$.

Step 2 The set S_k is formed from S_{k-1} by adding a vertex u not in S_{k-1} with the smallest label. Once u is added to S_k , we update the labels of all vertices not in S_k , so that $L_k(v)$, the label of the vertex v at the k th stage, is the length of the shortest path from a to v that containing vertices only in S_k .



Let v be a vertex not in S_k . To **update the label of v** , note that $L_k(v)$ is the shortest path from a to v containing only vertices in S_k ,

1. the shortest path from a to v containing only elements of S_{k-1}

Or

2. it is the shortest path from a to u at the $(k-1)$ st stage with the edge (u,v) added.

In other words,

$$L_k(v) = \min\{L_{k-1}(v), L_{k-1}(u) + w(u,v)\}$$



Algorithm 1 Dijkstra's Algorithm.

Procedure Dijkstra(G : weighted connected simple graph, with all weights positive)

{ G has vertices $a = v_0, v_1, \dots, v_n = z$ and weights $w(v_i, v_j)$ where $w(v_i, v_j) = \infty$ if $\{v_i, v_j\}$ is not an edge in G }

For $i := 1$ to n

$L(v_i) := \infty$

$L(a) := 0$

$S := \emptyset$

{the labels are now initialized so that the label of a is zero and all other labels are ∞ , and S is the empty set }



While $z \notin S$

Begin

$u :=$ a vertex not in S with $L(u)$ minimal

$S := S \cup \{u\}$

for all vertices v not in S

if $L(u) + w(u, v) < L(v)$ $L(v) := L(u) + w(u, v)$

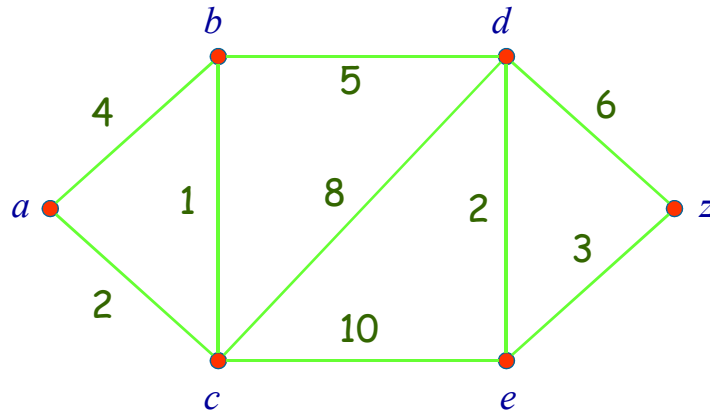
{this adds a vertex to S with minimal label and updates the labels of vertices not in S }

End { $L(z)$ = length of shortest path from a to z }

**➤ $previous(v) := u$
update $L(v)$ for all v not in S**



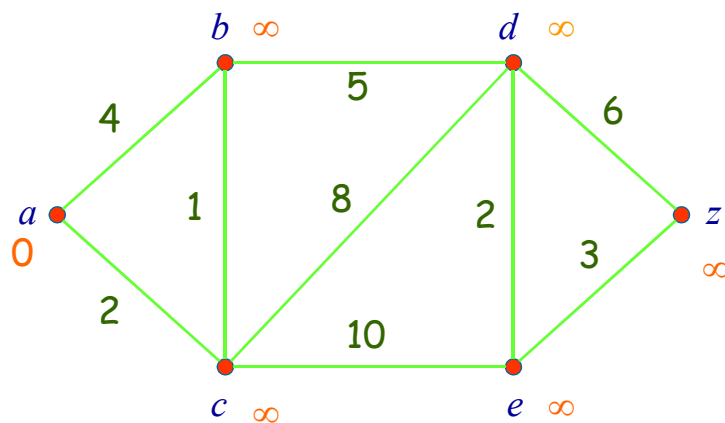
[[Example 1]] Find the length of the shortest path between a and z in the given weighted graph.



59



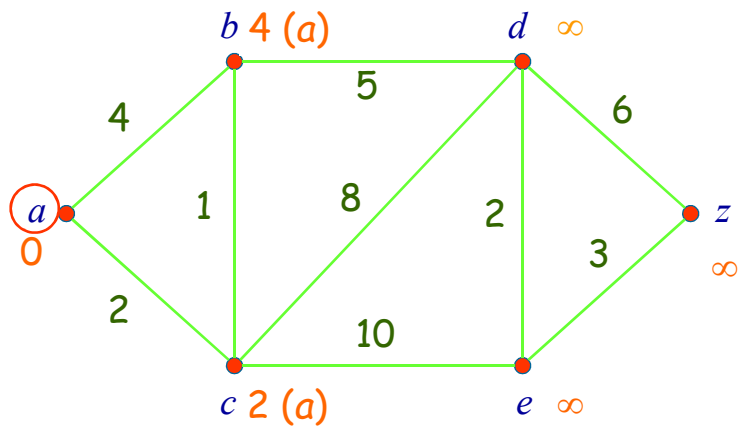
Step 0



60



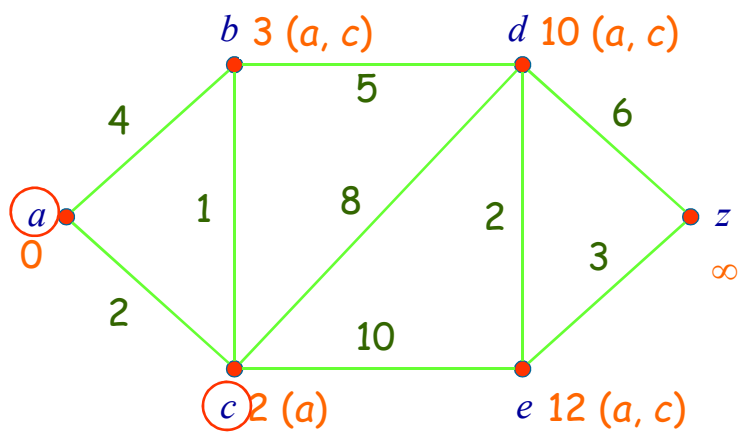
Step 1



61



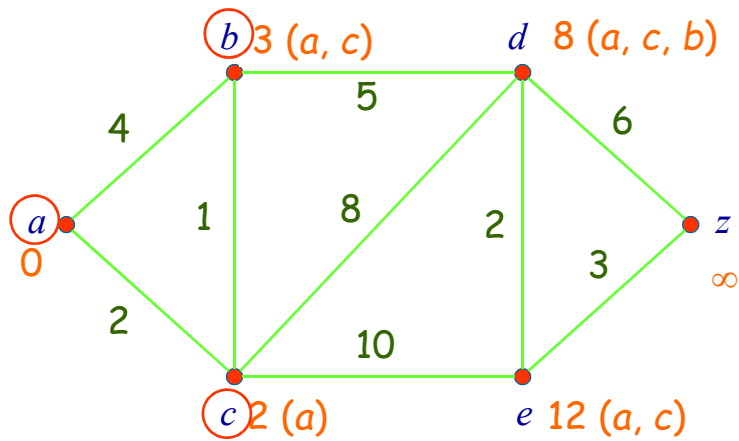
Step 2



62



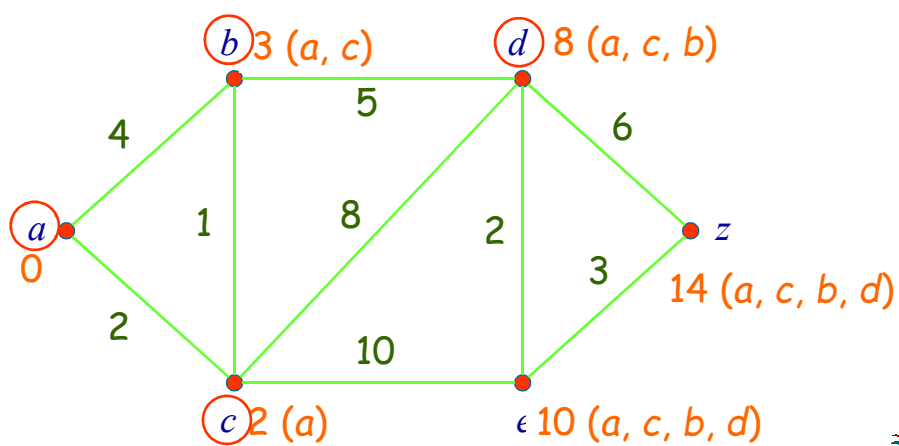
Step 3



63



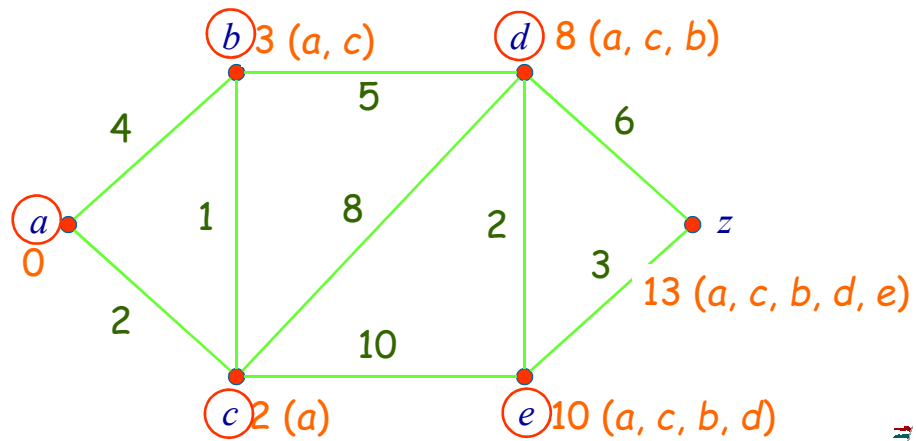
Step 4



64



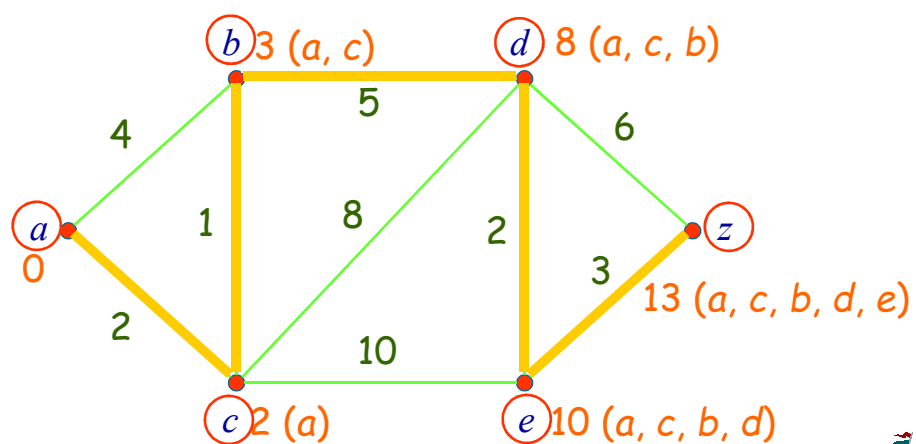
Step 5



65



Step 6

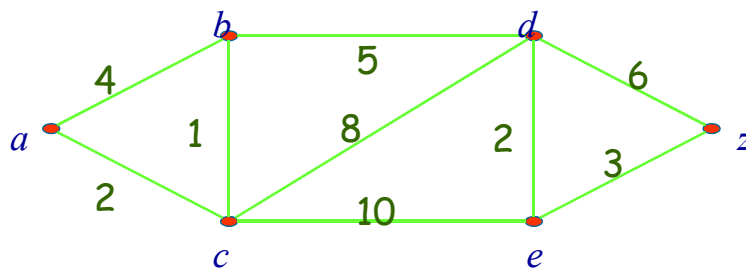


66



Remark:

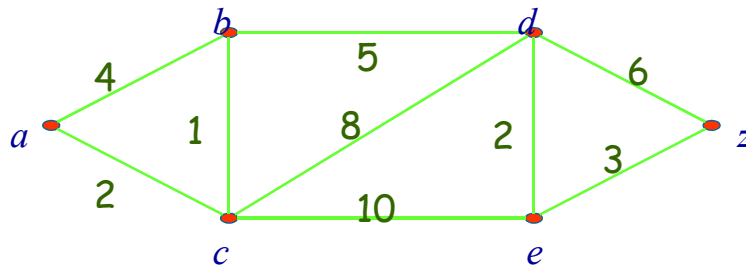
In performing Dijkstra's algorithm it is sometimes more convenient to keep track of labels of vertices in each step using a table.



Vertex	S	Link						
a								
b								
c								
d								
e								
z								



10.6 Shortest Path Problems

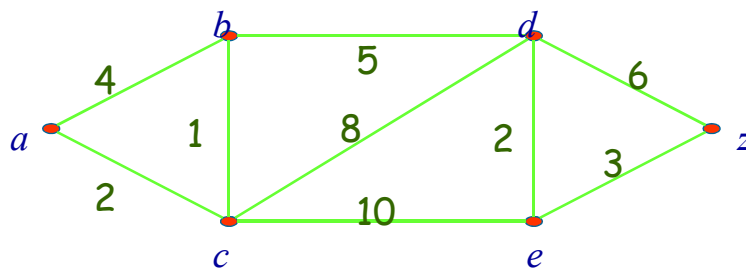


Vertex	S	Link	L_0					
a			0					
b			∞					
c			∞					
d			∞					
e			∞					
z			∞					

69



10.6 Shortest Path Problems

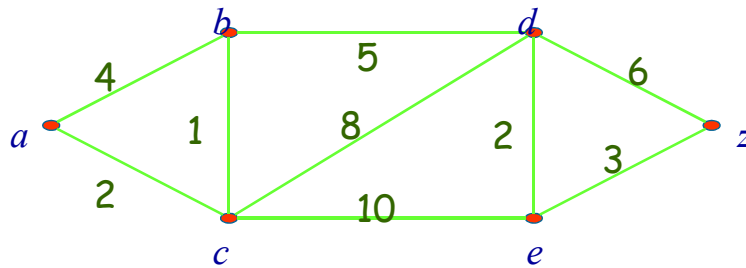


Vertex	S	Link	L_0	L_1				
a	1		0					
b		a	∞	4				
c		a	∞	2				
d			∞	∞				
e			∞	∞				
z			∞	∞				

70



10.6 Shortest Path Problems

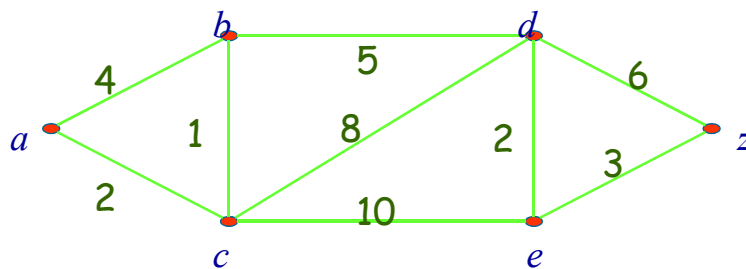


Vertex	S	Link	L_0	L_1	L_2			
a	1		0					
b		$a \rightarrow c$	∞	4	3			
c	1	a	∞	2				
d		$a \rightarrow c$	∞	∞	10			
e		$a \rightarrow c$	∞	∞	12			
z			∞	∞	∞			

71



10.6 Shortest Path Problems

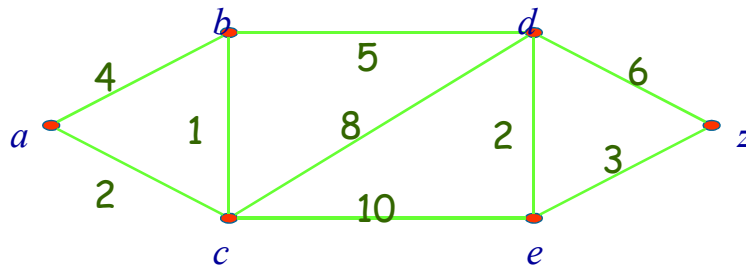


Vertex	S	Link	L_0	L_1	L_2	L_3		
a	1		0					
b	1	$a \rightarrow c$	∞	4	3			
c	1	a	∞	2				
d		$a \rightarrow c \rightarrow b$	∞	∞	10	8		
e		$a \rightarrow c$	∞	∞	12	12		
z			∞	∞	∞	∞		

72



10.6 Shortest Path Problems

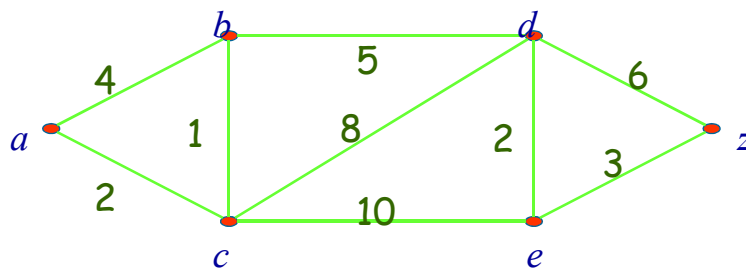


Vertex	S	Link	L ₀	L ₁	L ₂	L ₃	L ₄	
a	1		0					
b	1	a→c	∞	4	3			
c	1	a	∞	2				
d	1	a→c→b	∞	∞	10	8		
e		a→c→b→d	∞	∞	12	12	10	
z		a→c→b→d	∞	∞	∞	∞	14	

73



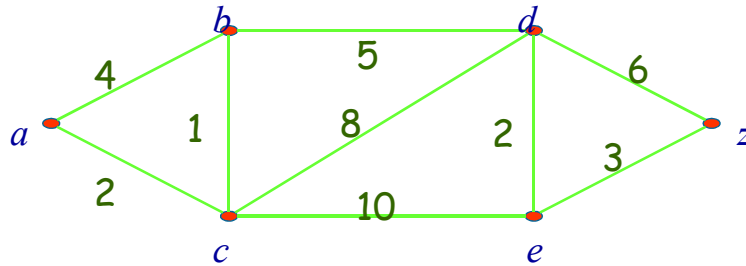
10.6 Shortest Path Problems



Vertex	S	Link	L ₀	L ₁	L ₂	L ₃	L ₄	L ₅
a	1		0					
b	1	a→c	∞	4	3			
c	1	a	∞	2				
d	1	a→c→b	∞	∞	10	8		
e	1	a→c→b→d	∞	∞	12	12	10	
z		a→c→b→d→e	∞	∞	∞	∞	14	13

74





Vertex	S	Link	L_0	L_1	L_2	L_3	L_4	L_5
a	1		0					
b	1	$a \rightarrow c$	∞	4	3			
c	1	a	∞	2				
d	1	$a \rightarrow c \rightarrow b$	∞	∞	10	8		
e	1	$a \rightarrow c \rightarrow b \rightarrow d$	∞	∞	12	12	10	
z	1	$a \rightarrow c \rightarrow b \rightarrow d \rightarrow e$	∞	∞	∞	∞	14	13

75



【 Theorem 1 】 Dijkstra's algorithm finds the length of a shortest path between two vertices in a connected simple undirected weighted graph. (只能正权重)

Proof:

Take as the induction hypothesis the following assertion:
At the k th iteration

- I. the label of every vertex v in S is the length of the shortest path from a to this vertex, and
- II. the label of every vertex not in S is the length of the shortest path from a to this vertex that contains only (besides the vertex itself) vertices in S .

(1) $k=0$

$S=\emptyset$, $L_0(a)=0$, $L_0(v)=\infty$

76



(2) Assume that the inductive hypothesis holds for the k th iteration.

Let v be the vertex added to S at the $(k+1)$ st iteration so that v is a vertex not in S at the end of the k th iteration with the smallest label.

■ **(I) holds at the end of the $(k+1)$ st iteration**

- ✓ The vertices in S before the $(k+1)$ st iteration are labeled with the length of the shortest path from a .
- ✓ v must be labeled with the length of the shortest path to it from a .

If this were not the case, at the end of the k th iteration there would be a path of length less than $L_k(v)$ containing a vertex not in S .



Let u be *the first* vertex not in S in such a path. There is a path with length less than $L_k(v)$ from a to u containing only vertices of S . This contradicts the choice of v .

■ **(II) is true.**

Let u be a vertex not in S after $k+1$ iteration.

A shortest path from a to u containing only elements of S either contains v or it does not.

- If it does not contain v , then by the inductive hypothesis its length is $L_k(u)$.
- If it does contain v , then it must be made up of a path from a to v of the shortest possible length containing elements of S other than v , followed by the edge from v to u . In this case its length would be $L_k(v) + w(v, u)$.



【 Theorem 2 】 Dijkstra's algorithm uses $O(n^2)$ operations (additions and comparisons) to find the length of the shortest path between two vertices in a connected simple undirected weighted graph.

Analysis:

- Use no more than $n-1$ iteration
- Each iteration,
 - using no more than $n-1$ comparisons to determine the vertex not in S_k with the smallest label
 - no more than $2(n-1)$ operations are used to update no more than $n-1$ labels



Floyd's Algorithm (find the distance $d(a, b) \forall a, b$)

Procedure *Floyd*(G : weighted simple graph)

$\{G$ has vertices $v_1, \dots, v_n$ and weights $w(v_i, v_j)$ with $w(v_i, v_j) = \infty$ if $\{v_i, v_j\}$ is not an edge $\}$

for $i := 1$ **to** n

for $j := 1$ **to** n

$d(v_i, v_j) := w(v_i, v_j)$

for $i := 1$ **to** n

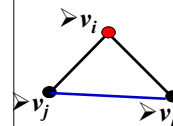
for $j := 1$ **to** n

for $k := 1$ **to** n

if $d(v_j, v_i) + d(v_i, v_k) < d(v_j, v_k)$

then $d(v_j, v_k) := d(v_j, v_i) + d(v_i, v_k)$

$\{d(v_i, v_j)$ is the length of a shortest path between v_i and $v_j\}$



➤对每个 v_i , 检查任何一对点目前的距离会不会因为改走 v_i 而变小。

➤This algorithm cannot be used to construct shortest paths.

可以有负权边但不能有负权回路



3. The Traveling Salesman Problem

✧ The traveling salesman problem

The traveling salesman problem asks for the circuit of minimum total weight in a weighted, complete, undirected graph that visits each vertex exactly once and returns to its starting point.

The equivalent problem:

Find a Hamilton circuit with minimum total weight in the complete graph.



✧ Solution of the traveling salesman problem

(1) A straightforward method

Examine all possible Hamilton circuits and select one of minimum total length.

The number of Hamilton circuits: $(n-1)!$

Since a Hamilton circuits can be traveled in reverse order, we need only examine $(n-1)!/2$ circuits to find the answer.

The time complexity: $n!$



(2) Approximation algorithm

These are algorithms that do not necessary produce the exact solution to the problem but instead are guaranteed to produce a solution that is close to an exact solution.

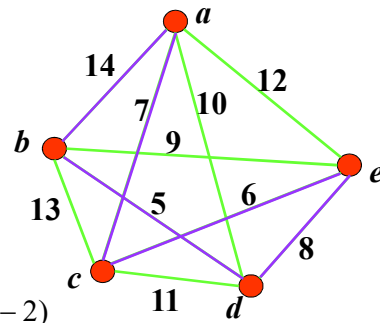
For example,

The length of this path: 40

The exact solution (the length is 37): a, c, e, b, d, a

The time complexity:

$$1 + 2 + 3 + \dots + (n - 2) = \frac{1}{2}(n - 1)(n - 2)$$

**Homework:**

Sec. 10.6 3, 17a), 26

